UNIVERSITY OF SCIENCE, VNU-HCM

Faculty of Information Technology

CSC14003 - Introduction to Artificial Intelligence

Project 2: Logic

Futoshiki Puzzles

Group Members

Nguyễn Khánh Toàn	MSSV: 24127252
Nguyễn Chí Tài	MSSV: 24127529
Nguyễn Ngọc Thiên	MSSV: 24127545
Phan Quang Tiến	MSSV: 24127250

Semester II (2025 - 2026)

Contents

1 Introduction **3**
- 1.1 Project Overview . . . 3
- 1.2 Objectives and Scope . . . 3
- 1.3 Team Information . . . 4

2 Project Planning and Task Distribution **4**
- 2.1 Work Breakdown . . . 4
- 2.2 Member Responsibilities . . . 4
- 2.3 Completion Percentage per Member . . . 4
- 2.4 Project Management Board . . . 4

3 Problem Formulation **4**
- 3.1 Puzzle Rules . . . 4
- 3.2 First-Order Logic Formalization . . . 4
  - 3.2.1 Predicate Vocabulary . . . 5
  - 3.2.2 Core Axioms . . . 5
  - 3.2.3 Skolemization and CNF Conversion . . . 5
- 3.3 From FOL to Inference . . . 5
- 3.4 Ground Knowledge Base and CNF Generation . . . 5

4 Algorithm Description **5**
- 4.1 Forward Chaining . . . 5
  - 4.1.1 Idea . . . 5
  - 4.1.2 Pseudocode . . . 5
  - 4.1.3 Illustration . . . 5
  - 4.1.4 Algorithm Analysis . . . 5
- 4.2 Backward Chaining (SLD Resolution) . . . 6
  - 4.2.1 Idea . . . 6
  - 4.2.2 Pseudocode . . . 6
  - 4.2.3 Illustration . . . 6
  - 4.2.4 Algorithm Analysis . . . 6
- 4.3 A* Search . . . 6
  - 4.3.1 Idea . . . 6
  - 4.3.2 Heuristic Function and Admissibility . . . 7

4.3.3 Pseudocode 8
4.3.4 Illustration 9
4.3.5 Algorithm Analysis 9
4.4 Brute-force and Backtracking 10
4.4.1 Idea 10
4.4.2 Pseudocode 10
4.4.3 Illustration 11
4.4.4 Algorithm Analysis 11
4.5 SAT Solver (Using SAT Library) 12
4.5.1 Idea 12
4.5.2 Constraint Encoding 12
4.5.3 Pseudocode 13
4.5.4 Illustration 13
4.5.5 Algorithm Analysis 14

5 Experiments and Evaluation 14
5.1 Experimental Setup 14
5.2 Input and Output Test Cases 15
5.3 Evaluation Metrics 15
5.3.1 Runtime 15
5.3.2 Memory Usage 15
5.3.3 Number of Inferences or Expansions 15
5.4 Comparative Analysis Across Algorithms 15
5.5 Self-evaluation Against Project Criteria 15

6 References 15

A Appendix 15
A.1 Pseudocode 15
A.2 Demo Video Links 16
A.3 Additional Tables and Charts 16
A.4 Submission Checklist 16

1. Introduction

1.1. Project Overview

This project implements a complete pipeline for solving *Futoshiki* logic puzzles in the context of CSC14003 — Introduction to Artificial Intelligence. A Futoshiki puzzle is an $N \times N$ grid where each row and column must contain numbers from 1 to N without repetition, while inequality symbols between adjacent cells must also be satisfied.

Our system is organized into clear modules: data I/O, domain modeling, knowledge-base construction, solver implementations, and a GUI for interaction and visualization. The codebase supports loading multiple input instances, running selected solving methods, and exporting results in a reproducible format. This report focuses on two representative approaches: A* search and SAT-based solving.

1.2. Objectives and Scope

The main objectives of this project are:

- Model Futoshiki constraints correctly and consistently for both search-based and logic-based solving.
- Implement and evaluate AI solving strategies with readable, modular, and reusable code.
- Build a practical interface for testing puzzle instances of different sizes and difficulties.
- Compare algorithm behavior through solution quality and computational efficiency.

Scope of this report:

- We present the overall project context.
- In the algorithm chapter, we provide detailed descriptions for only two methods: **A* Search** and **SAT Solver (using a SAT library)**.
- Other implemented methods are part of the codebase but are not expanded in detail here.

1.3. Team Information

The project was completed by four students from the Faculty of Information Technology, University of Science — VNU-HCM:

- Nguyễn Khánh Toàn — MSSV: 24127252
- Nguyễn Chí Tài — MSSV: 24127529
- Nguyễn Ngọc Thiên — MSSV: 24127545
- Phan Quang Tiến — MSSV: 24127250

The team collaborated on problem modeling, implementation, testing, and report writing, with task assignments summarized in the planning section.

2. Project Planning and Task Distribution

2.1. Work Breakdown

[Chèn nội dung tại đây]

2.2. Member Responsibilities

[Chèn nội dung tại đây]

2.3. Completion Percentage per Member

[Chèn nội dung tại đây]

2.4. Project Management Board

[Chèn nội dung tại đây]

3. Problem Formulation

3.1. Puzzle Rules

[Chèn nội dung tại đây]

3.2. First-Order Logic Formalization

[Chèn nội dung tại đây]

3.2.1 Predicate Vocabulary

[Chèn nội dung tại đây]

3.2.2 Core Axioms

[Chèn nội dung tại đây]

3.2.3 Skolemization and CNF Conversion

[Chèn nội dung tại đây]

3.3. From FOL to Inference

[Chèn nội dung tại đây]

3.4. Ground Knowledge Base and CNF Generation

[Chèn nội dung tại đây]

4. Algorithm Description

4.1. Forward Chaining

4.1.1 Idea

[Chèn nội dung tại đây]

4.1.2 Pseudocode

[Chèn nội dung tại đây]

4.1.3 Illustration

[Chèn nội dung tại đây]

4.1.4 Algorithm Analysis

[Chèn nội dung tại đây]

4.2. Backward Chaining (SLD Resolution)

4.2.1 Idea

[Chèn nội dung tại đây]

4.2.2 Pseudocode

[Chèn nội dung tại đây]

4.2.3 Illustration

[Chèn nội dung tại đây]

4.2.4 Algorithm Analysis

[Chèn nội dung tại đây]

4.3. A* Search

4.3.1 Idea

In this project, each search state is a partial Futoshiki assignment, where value 0 denotes an unfilled cell. The A* solver expands states according to

$$f(s) = g(s) + h(s),$$

with the following cost model:

- Each action assigns exactly one value to one empty cell, so each step has unit cost.
- $g(s)$ is the number of assignments already made from the initial state.
- $h(s)$ estimates the minimum number of assignments still needed to reach a complete solution.

The transition model assigns one value to one unfilled cell. To reduce branching, the solver selects the next variable by MRV (Minimum Remaining Values), i.e., the cell with the smallest candidate domain. Before evaluating child states, domains are filtered by row/column uniqueness and inequality constraints; optional AC-3 propagation is then applied to remove unsupported values.

This design combines informed search (A*) with CSP-style filtering (MRV + AC-3), making the exploration significantly more directed than naive uninformed expansion.

4.3.2 Heuristic Function and Admissibility

Let $U(s)$ be the number of unassigned cells in state s. We choose:

$$h(s) = U(s).$$

Equivalently, if N is board size and $A(s)$ is number of assigned cells, then

$$h(s) = N^2 - A(s).$$

Feasibility pruning with AC-3 is used as a separate consistency check: if propagation yields an empty domain, the state is discarded as inconsistent.

Why this heuristic is admissible:

- From a partial state with $U(s)$ empty cells, any complete solution must assign all those $U(s)$ cells.
- Each action can assign at most one cell.
- Therefore the true remaining cost $h^*(s)$ satisfies $h^*(s) \geq U(s) = h(s)$.

Hence $h(s)$ never overestimates the optimal remaining cost, so it is admissible.

Consistency (monotonicity):

- After one valid assignment $s \to s'$, we have $U(s') = U(s) - 1$.
- With unit step cost $c(s, s') = 1$:

$$h(s) = U(s) = 1 + U(s') = c(s, s') + h(s').$$

- So h is consistent as well.

4.3.3 Pseudocode

Algorithm 1 A* for Futoshiki (with MRV and AC-3)

```
Build initial state s_0 from puzzle
Validate givens; if invalid then return failure
Compute domains and heuristic for s_0
Initialize closed as an empty set
Initialize best_g[s_0]
Push s_0 into priority queue by key (f, h, −g)
while priority queue not empty do
    Pop best state s
    if s ∈ closed then
        continue
    end if
    Add s to closed
    if s is goal then return solved grid
    end if
    Select unassigned cell (r, c) by MRV
    for each value v in domain(r, c) do
        if assigning v is consistent then
            Create child state s′
            if s′ ∈ closed then
                continue
            end if
            Compute g(s′)
            if best_g[s′] exists and g(s′) is not better then
                continue
            end if
            Recompute domains (and AC-3 if enabled)
            if no empty domain in s′ then
                Set g(s′) = g(s) + 1, evaluate h(s′), f(s′) = g(s′) + h(s′)
                Update best_g[s′]
                Push s′ into priority queue
            end if
        end if
    end for
end while
return failure
```

4.3.4 Illustration

Consider a 4×4 instance. After loading givens and inequalities, the solver:

1. Initializes domains for all empty cells.
2. Runs consistency filtering; if AC-3 is enabled, unsupported values are removed transitively.
3. Chooses the most constrained empty cell by MRV.
4. Expands candidate assignments in priority order according to $f = g+h$.

Example progression:

- State s_0: 4 cells fixed from clues, several constrained inequalities.
- State s_1: assign value to the MRV cell (domain size 1 or 2), causing further domain reduction.
- State s_2: another forced assignment appears after propagation.
- Goal state: all cells filled, row/column uniqueness and inequalities all satisfied.

This illustrates how propagation shrinks the search tree before deep branching happens.

4.3.5 Algorithm Analysis

Let N be puzzle size (grid N^2 cells), and let b be average branching factor after filtering.

Time complexity (worst case):

- Search may still be exponential in the number of unassigned cells, roughly $O(b^d)$ with depth $d \leq N^2$.
- AC-3 propagation on each evaluated state adds polynomial overhead; practical runtime is dominated by number of expanded states.

Space complexity:

- A* stores open/closed states and domain data, so memory usage is higher than depth-first methods.
- In worst case, space is also exponential in search depth.

Practical behavior:

- The heuristic $h(s) = U(s)$ is admissible and easy to justify, which is suitable for the project requirement on heuristic correctness.
- Because this heuristic is simple, most practical speedup comes from MRV ordering and AC-3 pruning.

- The *closed* set avoids re-expanding visited states, and *best_g* avoids keeping inferior paths to the same state.
- The method is robust for medium-sized puzzles, but memory pressure can increase for larger or weakly constrained instances.

4.4. Brute-force and Backtracking

4.4.1 Idea

Brute-force is a straightforward approach that tries all possible assignments for the empty cells in the Futoshiki puzzle. For a puzzle of size $N \times N$, each empty cell can take a value from 1 to N. The algorithm generates all possible combinations and checks whether they satisfy the constraints (row, column, and inequality constraints). However, this approach is highly inefficient due to the exponential number of possibilities.

Backtracking is an optimized version of brute-force. Instead of generating all possible assignments, it builds the solution step by step and abandons a partial assignment as soon as it violates any constraint. At each step, the algorithm selects an empty cell, tries a valid value, and recursively proceeds to the next cell. If a conflict occurs, it backtracks and tries another value. This significantly reduces the search space compared to brute-force.

4.4.2 Pseudocode

Brute-force:

```
function BRUTE_FORCE(grid):
    empty_cells = list of all empty positions
    for each possible assignment of values to empty_cells:
        fill grid with assignment
        if is_valid(grid):
            return grid
    return failure
```

Backtracking:

```
function BACKTRACK(grid):
    if no empty cell:
```

```
        return grid

    (row, col) = find an empty cell

    for value from 1 to N:
        if is_valid(row, col, value):
            grid[row][col] = value

            result = BACKTRACK(grid)
            if result != failure:
                return result

            grid[row][col] = empty

    return failure
```

4.4.3 Illustration

Consider a 4×4 Futoshiki puzzle with some empty cells.

Brute-force will generate all possible combinations for the empty cells. For example, if there are k empty cells, it will try up to N^k assignments, checking each one against all constraints.

Backtracking, on the other hand, fills the grid step by step. Suppose it assigns a value that violates a constraint (e.g., duplicates in a row or breaks an inequality). Instead of continuing, it immediately backtracks and tries another value. This pruning avoids exploring invalid branches of the search tree, making it much more efficient in practice.

4.4.4 Algorithm Analysis

Brute-force Complexity: The time complexity is $O(N^k)$, where k is the number of empty cells. In the worst case (empty grid), this becomes $O(N^{N^2})$, which is infeasible for large N.

Backtracking Complexity: The worst-case time complexity is still exponential, but significantly reduced in practice due to pruning. The actual performance depends on how early invalid assignments are detected.

Space Complexity: Backtracking uses recursion, so the space complexity is $O(N^2)$ for the grid plus $O(N^2)$ for the recursion stack in the worst case.

Comparison:

- Brute-force explores all possibilities without pruning.
- Backtracking prunes invalid states early, making it much more efficient.
- Backtracking is the preferred method for solving constraint satisfaction problems like Futoshiki.

4.5. SAT Solver (Using SAT Library)

4.5.1 Idea

The SAT approach transforms the Futoshiki puzzle into a propositional satisfiability problem in CNF (Conjunctive Normal Form). Instead of searching directly over puzzle states, we encode all puzzle rules as logical clauses and let a SAT engine decide whether there exists a satisfying assignment.

In this project, CNF clauses are generated by the `FOLKB` module, then solved by PySAT (Glucose3). If SAT, the returned model is decoded back to an $N \times N$ grid; if UNSAT, the puzzle has no valid solution under given constraints.

4.5.2 Constraint Encoding

For each triple (i, j, v), define boolean variable $X_{i,j,v}$:

$$X_{i,j,v} = \text{true} \iff \text{cell } (i, j) \text{ has value } v.$$

Variable ID mapping used in code:

$$\text{ID}(i, j, v) = i \cdot N^2 + j \cdot N + v,$$

with $i, j \in [0, N-1]$, $v \in [1, N]$, so IDs are in $[1, N^3]$.

Main CNF groups:

- **Cell at least one value:** each cell must take some $v \in \{1, \ldots, N\}$.
- **Cell at most one value:** no cell can take two different values simultaneously.

- **Row uniqueness:** two different columns in same row cannot share the same value.
- **Column uniqueness:** two different rows in same column cannot share the same value.
- **Given clues:** pre-filled cells are encoded as unit clauses.
- **Inequality constraints:** for each adjacent pair with sign $<$ or $>$, all violating value-pairs are forbidden by binary clauses.

This encoding guarantees that any satisfying model corresponds to a valid Futoshiki solution.

4.5.3 Pseudocode

Algorithm 2 SAT-based Futoshiki Solver

```
Build CNF clauses from puzzle constraints using FOLKB
Create SAT solver instance (Glucose3)
Add all CNF clauses to solver
if solver returns UNSAT then
    return failure
end if
Get SAT model (set of literals)
Initialize empty grid N × N
for each positive literal ℓ in model do
    if 1 ≤ ℓ ≤ N^3 then
        Decode ℓ → (i, j, v)
        Set grid(i, j) ← v
    end if
end for
if any cell remains unassigned then
    return failure
else
    return solved grid
end if
```

4.5.4 Illustration

For a 4×4 puzzle:

1. The encoder creates variables $X_{i,j,v}$ for all $4^3 = 64$ combinations.

2. Clauses are generated for Latin constraints (row/column uniqueness), given clues, and inequality symbols.
3. Glucose3 finds a satisfying model, for example including literals that decode to assignments such as $(0, 0) = 2$, $(0, 1) = 4$, etc.
4. After decoding all positive in-range literals, the grid is fully reconstructed and validated as complete.

Compared with incremental state expansion, the SAT method delegates global consistency reasoning to a highly optimized conflict-driven SAT engine.

4.5.5 Algorithm Analysis

Encoding cost:

- Number of variables is N^3.
- Number of clauses is polynomial in N, dominated by pairwise exclusivity and inequality forbidding constraints.

Solving cost:

- SAT is NP-complete in general, so worst-case complexity is exponential.
- In practice, modern CDCL solvers (such as Glucose3) are often very effective on structured constraints like Futoshiki.

Space and implementation trade-offs:

- Requires storing full CNF and SAT internal structures.
- Provides a clean and extensible formulation: adding new logical constraints usually means adding new clause templates.

Practical behavior in this project:

- SAT solver gives deterministic feasibility checking (SAT/UNSAT).
- Performance is strong on many instances, especially when constraints are dense enough to guide conflict learning.

5. Experiments and Evaluation

5.1. Experimental Setup

[Chèn nội dung tại đây]

5.2. Input and Output Test Cases

[Chèn nội dung tại đây]

5.3. Evaluation Metrics

[Chèn nội dung tại đây]

5.3.1 Runtime

[Chèn nội dung tại đây]

5.3.2 Memory Usage

[Chèn nội dung tại đây]

5.3.3 Number of Inferences or Expansions

[Chèn nội dung tại đây]

5.4. Comparative Analysis Across Algorithms

[Chèn nội dung tại đây]

5.5. Self-evaluation Against Project Criteria

[Chèn nội dung tại đây]

6. References

[Chèn nội dung tại đây]

A. Appendix

A.1. Pseudocode

[Chèn nội dung tại đây]

A.2. Demo Video Links

[Chèn nội dung tại đây]

A.3. Additional Tables and Charts

[Chèn nội dung tại đây]

A.4. Submission Checklist

[Chèn nội dung tại đây]